

BACHELOR PROJECT PROTOTYPE

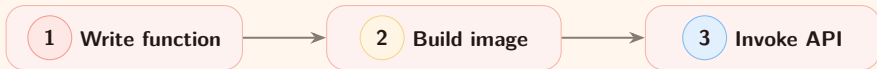
Fleeting Circus

A lightweight serverless platform
for Python functions

Upload code. Build once.
Invoke through an API.



The basic idea



Users bring Python code. The platform handles execution.



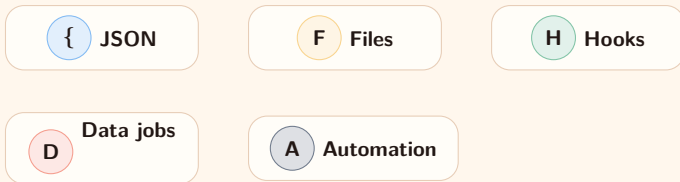
Why this project

- No manual server setup
- Repeatable function builds
- API calls for execution
- Results and artifacts tracked

~~code~~
result



Workloads



The prototype targets small Python services and short jobs.



Function code

```
def main(event, context):  
    name = event.get("name", "world")  
    print(f"hello {name}")  
    return {"message": f"Hello, {name}!"}
```

- event carries input
- context carries metadata
- return value becomes JSON

Dependencies

```
requirements.txt  
numpy  
requests==2.32.3  
pillow
```

install during build

Function image



Function contract

- Access mode
- Input file rules
- Expected output names
- Memory limit
- Function timeout
- Output size limits

The contract gives callers a predictable API and gives workers clear boundaries.



Access modes

Private	owner or admin JWT required
Token	caller sends a function invocation token
Public	caller can invoke without auth

Invocation tokens stay separate from login JWTs.



Synchronous invocation

- JSON-only functions
- No input files
- No declared output files
- Direct result when fast enough

```
POST /api/functions/4/invoke-sync/  
  
{"name": "Ada"}  
  
200 OK  
{"message": "Hello, Ada!"}
```



Asynchronous invocation

- Durable job path
- Supports files
- Supports polling
- Supports ZIP download

```
202 Accepted
{
  "invocation_id": 9274,
  "status": "queued",
  "read_token": "inv_..."
}
```



Result view

Result JSON

stdout

stderr

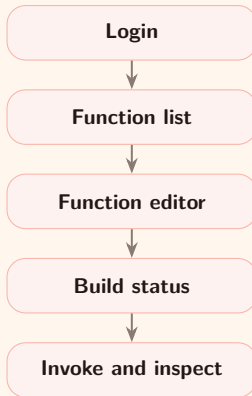
exit status

output paths

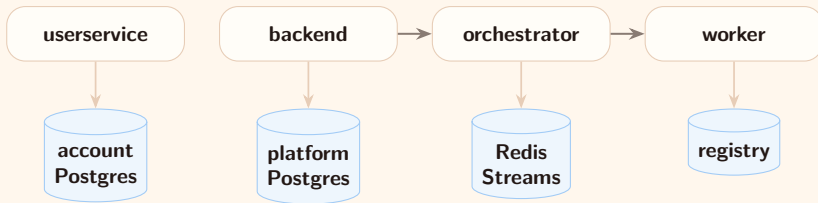
ZIP download

Simple responses stay small. Advanced responses expose diagnostics.

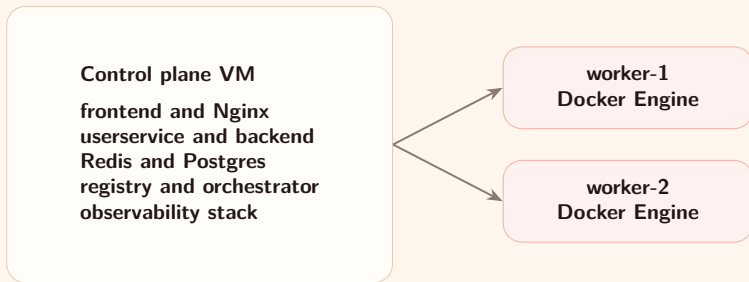
Frontend workflow



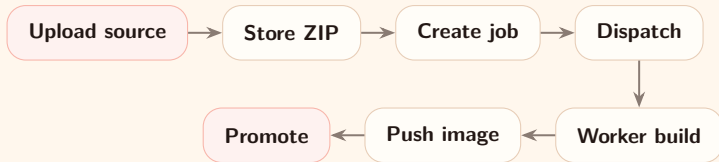
Main services



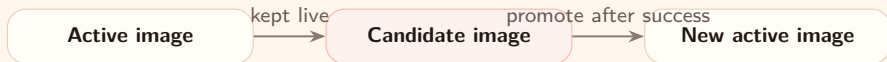
Current deployment



Build flow

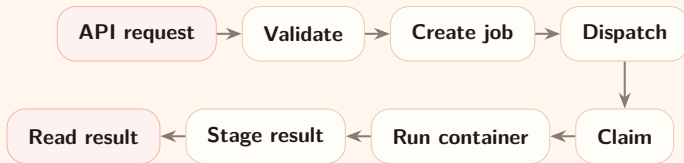


Safe rebuilds



Failed builds do not break the currently callable function.

Invocation flow





Files

- Input files are validated before execution
- Output filenames must be declared during setup
- Unexpected output files are rejected
- Inputs, outputs, and logs expire with the invocation



Staged artifacts

PROBLEM

A file can upload before the job is finalized.

STAGED

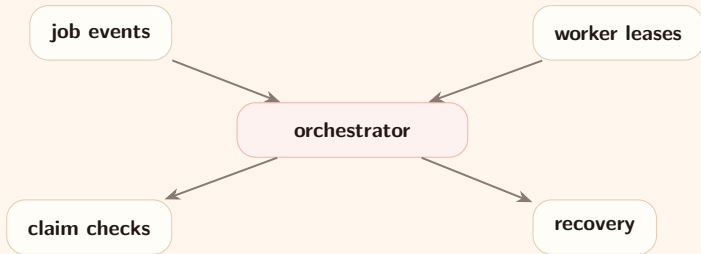
COMMITTED

VISIBLE

SOLUTION

Hide staged files until the finalizer commits them.

Orchestrator role



The orchestrator owns active V2 job state in Redis.



Worker role

- Read assigned Redis Stream
- Claim job through the orchestrator
- Download source or inputs from backend
- Run Docker build or function execution
- Report completion and maintain warm containers



Reliability model

Redis Streams

Attempt fencing

Heartbeats

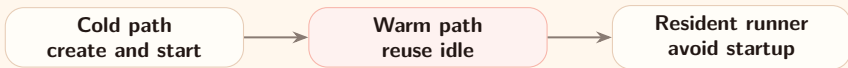
Lease recovery

Dead letter

Idempotency

Jobs either finish, recover, or move to a terminal failure state.

Warm containers



3918 ms

old warm no-op median

88 ms

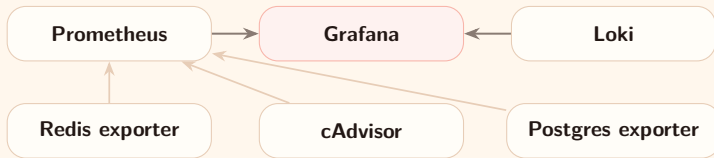
resident runner no-op median



Performance bottlenecks

- Docker container create and start
- Python module imports
- File copying and output handling
- Docker contention during concurrent bursts

Observability





Lessons from failures

- Backend-centered coordination became too heavy
- Docker copy and export paths were expensive
- Vite dev serving is weak for production
- Worker health missed consume-loop failure
- Private registry refs broke after network changes



Current state

- Working frontend
- Separate userservice
- V2 orchestrator
- Two distributed workers
- Token invocation
- Warm containers
- Artifact storage
- OpenAPI docs

Reliability and production hardening

- Recover worker consume-loop failures
- Harden worker and control plane networking
- Add stronger build resource limits
- Serve static frontend files
- Run longer failure and load tests

